重庆工程学院实验报告

课程名称（代码）：[20212144]Web 前端框架程序设计　**开课学院：**人工智能与计算机学院　**实验室：**7610

<table>
<tr><td>姓名</td><td>蒋静怡</td><td>学号</td><td>249980712</td><td>学院</td><td colspan="2">人工智能与计算机学院</td></tr>
<tr><td>年级、专业班级</td><td>24 计科 2 班</td><td>实验日期</td><td>2026.10.09</td><td>指导教师</td><td colspan="2">周瑞锋</td></tr>
<tr><td rowspan="5">实验项目名称</td><td colspan="3" rowspan="5">实验三：路由配置与页面导航</td><td rowspan="5">类别</td><td>验证性</td><td>☑</td></tr>
<tr><td>演示性</td><td>☐</td></tr>
<tr><td>综合性</td><td>☐</td></tr>
<tr><td>设计性</td><td>☐</td></tr>
<tr><td>创新性</td><td>☐</td></tr>
<tr><td>成绩</td><td colspan="6"></td></tr>
<tr><td>实验目的</td><td colspan="6">1. 能够安装并注册 Vue Router，理解单页面应用中 URL、路由规则与页面组件之间的对应关系。
2. 能够配置命名路由、导航高亮、404 通配符路由以及 Hash、History 两种路由模式。
3. 能够配置动态路由和嵌套路由，使用 useRoute() 获取路由参数。
4. 能够使用 useRouter() 实现 push、replace、go(-1) 等编程式导航，并说明其适用场景。
5. 能够使用 redirect、meta 和 beforeEach 完成默认页面跳转、页面标题更新及后台管理页面的访问控制。</td></tr>
<tr><td>实验任务与要求</td><td colspan="6">基本要求（必做）
1. 根据个人主页内容合理设计并配置路由，能够通过 URL 访问主要页面，顶部导航区域实现导航高亮显示。
2. 新增项目详情页，并结合项目数据合理使用动态路由与路由参数。
3. 新增身份认证页面和后台管理页面，用于修改个人简介、项目信息或其他适合管理的内容。
4. 在项目中综合使用 RouterView、RouterLink、命名路由、嵌套路由、编程式导航、重定向、路由元信息或路由守卫等知识。
5. 公开展示页面可直接访问；内容修改入口应与普通访客浏览功能合理区分。
6. 浏览器标题需要跟随页面展示的内容发生变化；
7. 页面内容、页面数量、组件划分、管理功能、布局方式和视觉风格可根据</td></tr>
</table>

	实际需求自主设计。 **拓展要求（选做）** 1. 根据个人主页内容，自主增加项目新增、删除、表单校验、保存提示或取消修改等功能，优化内容管理体验。 2. 完成其他基本要求以外的任务，合理即可。
实验原理（技术）	**1. SPA 与 Vue Router** SPA 只在首次访问时加载完整页面，之后通过 JavaScript 切换页面组件。Vue Router 根据浏览器 URL 匹配对应的路由规则，并将页面组件显示在 RouterView 中，从而实现不刷新整个页面的导航效果。 **2. 路由组件与普通组件** 路由组件通常代表能够通过独立 URL 访问的完整页面，统一放在 views 目录；普通组件是页面内部可复用的局部结构，通常放在 components 目录。合理区分两者有助于明确页面层级和组件职责。 **3. 路由配置项** 路由表通过 path 定义访问地址，通过 name 为路由提供稳定标识，通过 component 指定需要显示的页面组件。还可以使用 redirect 设置默认跳转，使用 meta 保存页面标题和访问权限等附加信息。 **4. 动态路由与嵌套路由** 动态路由使用:id 等参数表示可变化的路径片段，使多个项目详情共用同一个页面组件。嵌套路由用于组织具有共同页面结构的多个子页面。父路由组件负责共享标题、导航或整体布局，子路由组件显示在父组件内部的 RouterView 中。 **5. 编程式导航** 当页面跳转需要结合数据保存、条件判断或用户操作时，可以使用 useRouter() 进行编程式导航。push()用于普通跳转并保留当前历史记录，replace()用于替换当前记录，go(-1)用于返回上一页。 **6. 全局前置守卫 beforeEach** beforeEach 会在每次路由切换前执行，可以读取目标路由的 meta 信息，统一更新页面标题或判断页面访问条件。本实验中，公开展示页面直接放行，仅在访问内容修改页面时检查所有者身份。

实验仪器设备（环境条件）	个人 PC 机一台、VS Code、浏览器等
实验操作方法和步骤	1. 配置路由：创建并注册 Vue Router，根据页面关系设计基础路由、动态路由和嵌套路由。 2. 完成导航：根据实际需要使用 RouterLink 或 useRouter() 实现页面跳转、返回和默认页面重定向。 3. 完善管理功能：设计个人简介或项目信息的编辑页面，并使用路由元信息和守卫控制内容修改入口。 4. 自主完善：项目数量、详情内容、后台管理结构、页面布局、视觉风格和交互方式可根据个人主页需求自主设计。 5. 运行验证：检查 URL、页面切换、动态参数、嵌套路由、导航历史和异常地址处理是否符合预期。

实验记录与处理	1. 安装并注册 Vue Router
	`/* ========== 应用入口 main.js ========== */`
	`import './assets/main.css'`
	
	`import { createApp } from 'vue'`
	`import App from './App.vue'`
	`import router from './router/index.js'`
	
	`// 创建应用实例并注册路由`
	`createApp(App).use(router).mount('#app')`
	
	代码作用：通过 createApp 创建应用实例，使用 app.use(router)注册路由插件，使整个应用支持路由导航。

2. 配置路由表与基础路由

src > router > JS index.js > routes

```
/* ============ 路由配置 ============ */
import { createRouter, createWebHashHistory, createWebHistory } from 'vue-router'
import { isLoggedIn } from '../store.js'

// 路由级组件：使用动态导入实现懒加载
const HomeView = () => import('../views/HomeView.vue')
const ProjectsView = () => import('../views/ProjectsView.vue')
const ProjectListView = () => import('../views/ProjectListView.vue')
const ProjectDetailView = () => import('../views/ProjectDetailView.vue')
const AboutView = () => import('../views/AboutView.vue')
const NotFoundView = () => import('../views/NotFoundView.vue')
const LoginView = () => import('../views/LoginView.vue')
const AdminView = () => import('../views/AdminView.vue')
const ProfileEditView = () => import('../views/admin/ProfileEditView.vue')
const ProjectEditView = () => import('../views/admin/ProjectEditView.vue')

// 路由表
const routes = [
// 根路径重定向到首页
  {
    path: '/',
    redirect: '/home'
  },
  // 首页
  {
    path: '/home',
    name: 'Home',
    component: HomeView,
    meta: { title: '首页' }
  },
  // 项目页（父路由：嵌套路由布局）
  {
    path: '/projects',
    name: 'Projects',
    component: ProjectsView,
    redirect: '/projects/list',
    meta: { title: '项目展示' },
    children: [
      // 项目列表（嵌套子路由）
      {
        path: 'list',
        name: 'ProjectList',
        component: ProjectListView,
        meta: { title: '项目列表' }
      },
      // 项目详情（动态路由参数 :id）
      {
        path: 'detail/:id(\\d+)',
        name: 'ProjectDetail',
        component: ProjectDetailView,
        meta: { title: '项目详情' }
      }
    ]
  },
```

代码作用：定义 routes 数组，每个路由包含 path 路径、name 命名路由、component 组件、meta 元信息。使用 createRouter 创建路由实例，采用 Hash 模式。

3. 配置动态路由与项目详情页

views ↗ ▼ ProjectDetailView.vue ↗ { } script setup

```
<script setup>
/* ============ 项目详情视图 ProjectDetailView ============
   动态路由组件：对应路由 /projects/detail/:id
   使用 useRoute() 获取路由参数，展示对应项目的详细信息 */
import { computed } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { projects } from '../store.js'
import StatusBadge from '../components/StatusBadge.vue'

const route = useRoute()
const router = useRouter()

const projectId = computed(() => Number(route.params.id))

// 根据 id 查找对应项目
const project = computed(() => {
return projects.find((p) => p.id === projectId.value)
})
```

代码作用：/projects/detail/:id(\\d+)匹配数字 id 的项目详情地址，Number()转换后从项目数据中查找对应项目。

4. 配置嵌套路由

```
import { useRouter } from 'vue-router'
import { isLoggedIn } from '../store.js'

const router = useRouter()

// 退出登录
const handleLogout = () => {
  isLoggedIn.value = false
  console.log('已退出登录')
  // replace 到首页，不留后台历史记录
  router.replace({ name: 'Home' })
}
</script>

<template>
  <div class="admin-view">
    <div class="admin-layout">
      <!-- 左侧管理菜单 -->
      <aside class="admin-sidebar">
        <h3 class="sidebar-title">后台管理</h3>
        <nav class="sidebar-nav">
          <RouterLink to="/admin/profile" class="sidebar-link">个人简介编辑</RouterLink>
          <RouterLink to="/admin/projects" class="sidebar-link">项目信息管理</RouterLink>
        </nav>
        <button class="btn btn-default logout-btn" @click="handleLogout">退出登录</button>
      </aside>

      <!-- 右侧内容区：嵌套路由出口 -->
      <main class="admin-content">
        <RouterView />
      </main>
    </div>
  </div>
</template>
```

代码作用：通过 children 配置项组织父子路由关系，父路由组件提供共享布局，子路由内容渲染在父组件内部的 RouterView 出口中。

5.实现编程式导航

```
import { ref } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { isLoggedIn } from '../store.js'

const route = useRoute()
const router = useRouter()

// 登录表单
const form = ref({
  username: '',
  password: ''
})
const errorMsg = ref('')

// 登录提交：模拟认证
const handleLogin = () => {
  if (!form.value.username || !form.value.password) {
    errorMsg.value = '请输入用户名和密码'
    return
  }
  // 模拟登录成功：任意非空账号密码均可登录
  isLoggedIn.value = true
  errorMsg.value = ''
  console.log('登录成功：', form.value.username)

  // 登录成功后跳回原目标页面（从 query.redirect 获取）
  const redirect = route.query.redirect || '/admin'
  // 使用 replace 避免登录页留在历史记录中
  router.replace(redirect)
}
</script>
```

代码作用：router.push()用于普通页面跳转并保留历史记录；router.replace()替换当前历史记录用于登录后跳转和 404 返回；router.go(-1)返回上一页。

6. 配置路由守卫与 meta 元信息

```
router.beforeEach((to, from, next) => {
  // 根据 meta.title 更新浏览器标签页标题
  document.title = to.meta.title ? `${to.meta.title} - 个人主页` : '个人主页'

  // 路由守卫：检查是否需要登录权限
  if (to.meta.requiresAuth && !isLoggedIn.value) {
    // 未登录 → 重定向到登录页，并记录目标路径
    next({ name: 'Login', query: { redirect: to.fullPath } })
  } else {
    next()
  }
})

export default router
```

代码作用：每次路由切换前执行，根据 to.meta.title 更新浏览器标签页标题；检查 to.meta.requiresAuth，未登录时重定向到登录页并记录 redirect 参数。

7. 改造导航组件

```
<template>
  <header class="page-header">
    <div class="header-inner">
      <!-- 品牌名称：点击返回首页 -->
      <RouterLink to="/home" class="brand-link">
        <h1 class="brand">{{ title }}</h1>
      </RouterLink>

      <!-- 导航菜单：RouterLink 自动处理跳转和高亮 -->
      <nav class="nav">
        <RouterLink
          v-for="item in navItems"
          :key="item.name"
          :to="item.path"
          class="nav-link"
        >
          {{ item.label }}
        </RouterLink>
      </nav>

      <!-- 主题切换按钮 -->
      <button class="btn btn-theme" @click="emit('toggle-theme')">
        {{ theme === 'light' ? '切换暗色' : '切换亮色' }}
      </button>
    </div>
  </header>
</template>
```

代码作用：RouterLink 自动为当前激活的链接添加 router-link-active 和 router-link-exact-active 类，配合 CSS 实现当前页面导航项高亮显示。

AI 工具使用说明（必填）

序号	AI 工具名称	使用目的	关键提示词	AI 回复关键内容	采纳比例
1	豆包	路由配置代码生成	帮我配置 Vue Router，包含首页、项目页、动态路由详情、嵌套路由和路由守卫	生成了完整的路由配置文件结构，包含路由表、路由守卫和导航配置	采纳率约 90%
2	豆包	视图组件代码编写	根据实验二的项目数据，创建项目详情页、后台管理页等视图组件	生成了各视图组件的基础结构和样式，人工调整后使用	采纳率约 85%
3	豆包	问题排查与调试	路由跳转后页面内容不更新,Hash 模式下嵌套路由不生效	定位了组件导入和路由配置的问题，指导修复	采纳率约 95%

实验结果及分析

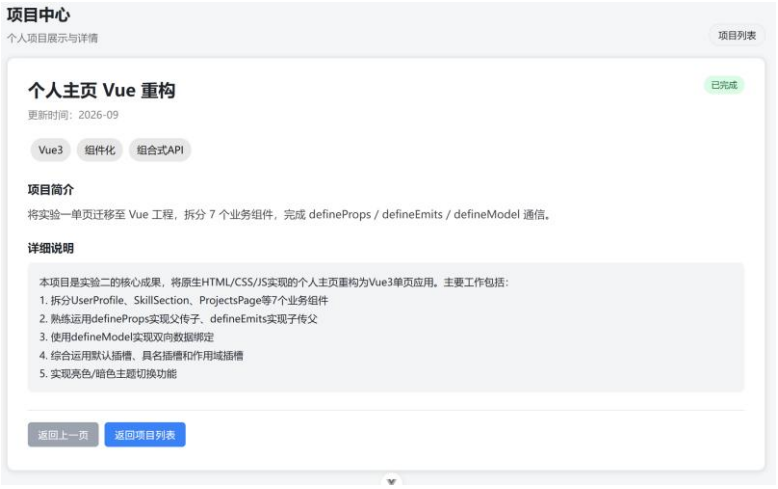
1. 基础路由功能验证

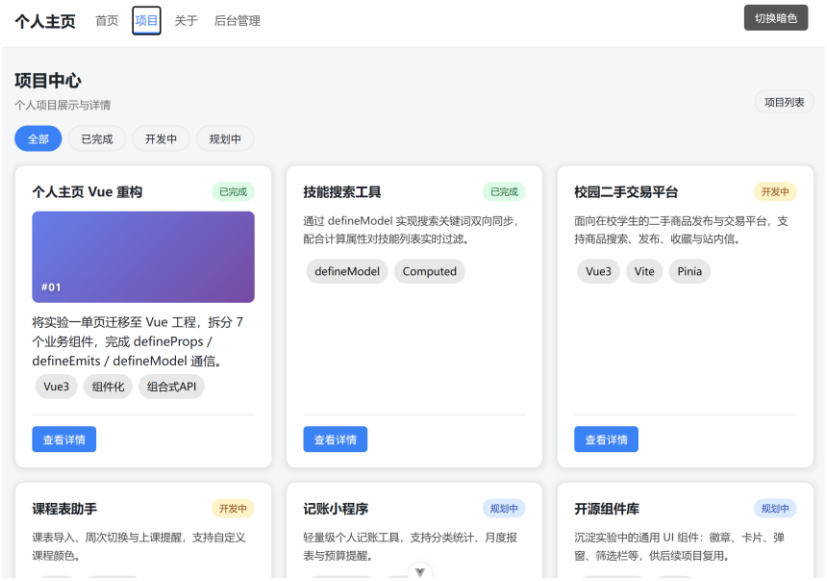
通过浏览器地址栏访问/home、/projects/list、/about 等路径，页面正常切换且无刷新。顶部导航栏当前所在项高亮显示。



2. 动态路由验证

访问/projects/detail/1 显示第一个项目详情，访问/projects/detail/3 显示第三个项目详情，参数正确传递并渲染对应内容。访问不存在的 id 时显示"未找到该项目"提示。





3. 嵌套路由验证

项目中心页作为父路由，左侧显示项目中心标题，右侧根据子路由显示项目列表或项目详情。后台管理页左侧显示管理菜单，右侧显示对应编辑表单。



4. 编程式导航验证

点击项目详情页的"返回上一页"按钮正确返回列表页；点击"返回项目列表"按钮正确跳转；404 页面点击"返回首页"正确跳转。



5. 路由守卫与页面标题验证



未登录状态下访问/admin 自动跳转到登录页，登录成功后自动跳回原目标页面。浏览器标签页标题随页面切换自动更新（如"首页 - 个人主页"）。

6. 拓展功能验证

后台管理页支持新增项目、删除项目，个人简介编辑表单可修改并保存，操作有成功提示。



	结果分析与实验收获： 本次实验成功实现了 Vue Router 的各项功能。基础路由和命名路由实现了页面间的基本导航；动态路由通过:id 参数实现了一条路由匹配多个项目详情页面；嵌套路由通过 children 配置实现了父子页面的层级关系，父路由提供共享布局，子路由提供内容；编程式导航通过 useRouter()实现了灵活的页面跳转控制；路由守卫 beforeEach 实现了统一的页面标题更新和登录权限校验。 通过本次实验，我深入理解了单页面应用（SPA）的工作原理和 Vue Router 的核心机制。掌握了路由配置、动态路由、嵌套路由、编程式导航和路由守卫的使用方法，理解了路由组件（views 目录）与普通组件（components 目录）的区别和职责划分。在实验过程中也遇到了 Hash 模式下路由跳转后页面不更新等问题，通过查阅文档和调试逐步解决，加深了对路由原理的理解。本次实验为后续开发复杂的单页面应用打下了坚实的基础。